Chapter 3: Implementation and Results to Date

LeakLens, Software Engineering Minor Project, Semester V. Progress report of 12 November 2026.

Project	LeakLens: Static and Dynamic Detection of Data Leakage in Machine Learning Code
Theme	Software Engineering (Minor Project, Semester V)
Submitted by	Srijeet Prasad Banerjee, B.Tech IT, 3rd Year
Supervisor	Prof. Tridib Chakraborty
Institute	Guru Nanak Institute of Technology, Kolkata
Repository	https://github.com/bytes06runner/LeakLens (all results below are logged in results/)

3.1 Summary

Chapters 1 and 2 proposed a tool that finds data leakage in Python ML code, confirms it on the data, and measures how much each leak inflates the reported score. On 29 September the repository held the taxonomy, the specification, a sandbox and a labelled benchmark, and **no detector**. This chapter reports what exists on 12 November: a static analyser for all six leakage types, a dynamic (instrumented-run) pass, per-finding impact estimation, a command-line tool with JSON and SARIF output, a frozen evaluation protocol, and the first held-out results.

Main result, on the held-out TEST half of the injected benchmark (79 files, 73 labelled leaks; scored once, after the protocol was frozen; the tool's rules were never tuned on these files): reading the code alone, LeakLens finds 59 of 73 leaks (81%); with the instrumented run, 65 of 73 (89%). It reports 0 false positives and nothing on the 9 files that have no leak.

Three cautions belong next to that number. (1) In 4 of the 7 group leaks it finds on TEST, the tool points at the right line but names the wrong column (a visit number instead of the patient identifier); only 3 of 7 are right for the right reason, and the impact estimate for those 4 is wrong. (2) The benchmark files are generated by this project; they are simpler than real notebooks and contain the kinds of leak the tool was built for. (3) No real notebook has been labelled, so there is no result on real code. On the companion project's own code the tool found nothing (section 3.8).

3.2 Progress against the plan

Phase	Deliverable	29 September	12 November
0	Setup, novelty check, baseline runnability	done	done; novelty search repeated today (3.8)
1	Taxonomy of six types, specification, sandbox	done	done
2	Benchmark: 175 injected files (162 leaks) and 150 real notebooks, DEV/TEST split frozen before any rule	injected done; real set sampled, unlabelled	unchanged: real notebooks still need two human labellers
3	Evaluation protocol	not started	**frozen** (version 1.0, approved by the project owner)
4	Static analyser	not started	**built**: all six types, 150+ rule tests
5	Dynamic analyser	not started	**built**: confirms, refutes, and finds data-only leaks
6	Impact estimation	not started	**built** for six kinds of fix; about half of the leaks have no automatic fix yet
7	Evaluation on TEST, baselines, case study	not started	**injected TEST scored once**; one baseline run (Yang et al.); CalibraEEG case study: negative result; real set and other baselines pending
8	Mining study of public notebooks	not started	not started
9	Release (PyPI, pre-commit, GitHub Action)	not started	CLI, JSON, SARIF done; packaging not started; name clash found (3.8)

3.3 What was built

Component	What it does
Ingest	Reads .py and .ipynb files into one module with a map back to cell and line. Uses execution order when every cell has one, else cell order, and records which. Files that do not parse are reported, never silently skipped.
Static analyser	One pass of abstract interpretation over pandas, NumPy, scikit-learn and imbalanced-learn code. For every variable it tracks which table the rows came from, which side of which split they are on, which columns they hold and what has leaked into them. Rules fire where rows are split, a model is fitted and scored, or a best score is selected. The analysed code is never run.
Dynamic pass	Runs the script with scikit-learn instrumented. At every split it measures, fold by fold, identical rows on both sides, entity values on both sides, and training rows dated after evaluation rows. Entity and time columns are found in the script's own variables, so **the user declares nothing**.
Impact estimation	Re-runs the user's own script with one call site changed (group-aware or chronological split, evaluation rows removed from a fit, a transformer fitted on training rows only, duplicates or one feature removed) and reports the change in the metric the script itself prints.
Reporting	Terminal output, JSON (validated against a schema), SARIF 2.1.0 for GitHub code scanning, CI exit codes, inline suppression comments.
Evaluation	One matching rule and bootstrap confidence intervals shared by every tool; a runner that refuses TEST until the protocol is frozen and refuses a second TEST run unless it is logged.

About 5,200 lines of Python in the package and 1,500 lines of tests (360 tests pass, including property-based tests that the analyser never crashes and is deterministic).

3.4 A worked example

A DEV pipeline loads a table of patients from 12 hospital sites, keeps four marker columns and calls `train_test_split`. The code never mentions the column `site`, so no static tool can see a problem. LeakLens output for that file (static + dynamic + impact):

```
line 17 [high/high] group (LL-GRP-004) run: confirmed test_auc 0.6865 -> 0.3016 with the fix
The data has a column `site` that the split ignores: 12 of its values are on both sides, and 100% of the evaluation rows belong to an entity that is also in the training rows.
fix: Split by entity: use GroupShuffleSplit with groups=`site` (same folds, test size and seed).
```

This is the sentence Chapter 1 asked for ("line 42 lets the same patient into both training and test, and fixing it drops your score from X to Y"). The size of the drop is a property of our synthetic data, not evidence about real studies, and section 3.7 shows that this kind of estimate is the least accurate one.

3.5 Detection results on the held-out TEST split

Injected benchmark, TEST half: 79 files from 6 base pipelines that are different from the 7 used for development, 73 labelled leaks, 9 files without a leak. A finding counts only if it has the right type and overlaps the labelled lines in the right cell; findings of medium or high confidence are counted. Intervals: bootstrap over files, 2,000 resamples. With so few leaks per type the intervals are wide.

Leakage type	Labelled leaks	Static: found	Static: recall (95% CI)	Static + dynamic: found	Static + dynamic: recall (95% CI)	False positives (either mode)
Preprocessing	22	21	0.95 (0.84 to 1.00)	21	0.95 (0.84 to 1.00)	0
Overlap / duplicates	9	7	0.78 (0.44 to 1.00)	9	1.00 (1.00 to 1.00)	0
Multi-test	6	5	0.83 (0.43 to 1.00)	5	0.83 (0.43 to 1.00)	0
Group (entity)	7	3	0.43 (0.00 to 0.83)	7	1.00 (1.00 to 1.00)	0
Temporal	25	20	0.80 (0.62 to 0.95)	20	0.80 (0.62 to 0.95)	0
Target-derived	4	3	0.75 (0.00 to 1.00)	3	0.75 (0.00 to 1.00)	0
All types	**73**	**59**	**0.81**	**65**	**0.89**	**0**

Precision is 1.00 for every type (no false positive). The files are synthetic and short; this will not hold on real code.

- **What was missed (8 leaks).** Five temporal leaks (a scaler or statistic fitted over a whole time series) were reported only at low confidence, and one multi-test leak was not reported, all in one pipeline that drops its last row with `data.iloc[:-1]`: the analyser mistook that slice for a train/test split. One target-encoding leak and one target-derived feature were missed because the feature list was built with a `startswith` filter the analyser cannot resolve, and it reports nothing rather than guess. With low-confidence findings included, temporal recall is 1.00.

- **Group leakage: right line, wrong column in 4 of 7.** For the EEG-style pipeline the entity comes from file names and is found from the code alone (3 of 3). For the patient-visits pipeline the static pass made a low-confidence guess on a column called `visit`; the run then "confirmed" it, because visit numbers repeat on both sides of any split, and never looked at `patient_id`. Under the frozen matching rule (type and line) these count as found, and the table shows 7 of 7. Counted strictly, group recall on TEST is 3 of 7.
- **Temporal categories:** common 5 of 5, differentiating 7 of 7 (default shuffle of train_test_split, unshuffled KFold, a time column with an uninformative name), hard 8 of 13. The temporal baseline (scicode-lint) has not been run.
- **Speed.** Static analysis: median 2.6 ms per file (laptop, Apple M5). An instrumented run costs one execution of the script (median 1.3 s).
- **These three causes have not been fixed.** Fixing them now and re-running TEST would be a second look at the test set; it is logged as an open decision, not done.

3.6 Results on the DEV split (the files the rules were written on)

Leakage type	Labelled leaks	Static: found	Static: recall (95% CI)	Static + dynamic: found	Static + dynamic: recall (95% CI)	False positives (either mode)
Preprocessing	30	30	1.00 (1.00 to 1.00)	30	1.00 (1.00 to 1.00)	0
Overlap / duplicates	10	8	0.80 (0.50 to 1.00)	10	1.00 (1.00 to 1.00)	0
Multi-test	8	8	1.00 (1.00 to 1.00)	8	1.00 (1.00 to 1.00)	0
Group (entity)	10	3	0.30 (0.00 to 0.62)	10	1.00 (1.00 to 1.00)	0
Temporal	25	25	1.00 (1.00 to 1.00)	25	1.00 (1.00 to 1.00)	0
Target-derived	6	6	1.00 (1.00 to 1.00)	6	1.00 (1.00 to 1.00)	0
All types	**89**	**80**	**0.90**	**89**	**1.00**	**0**

DEV: 96 files, 89 leaks, 10 files without a leak, no false positive. These numbers are optimistic by design and are shown only to make the gap to TEST visible: 100% on DEV against 89% on TEST with the run. The static analyser also ran on the 71 real DEV notebooks without a crash; nobody has labelled them, so no precision or recall can be stated for real code.

3.7 Impact estimation: coverage and accuracy

For an injected file the true inflation is known: the metric of the leaky file minus the metric of its hand-written correct version, on the same data. LeakLens's estimate is the metric of the original run minus the metric when the run is repeated with the finding's fix. On TEST both are averaged over five data seeds. In every one of the 395 TEST runs the instrumented script printed exactly the metric the plain script prints, so the instrumentation does not disturb the pipeline.

Coverage on TEST: for how many correctly located leaks an estimate could be produced

Leakage type	Estimated (TEST)	No estimate (TEST)	Why no estimate
Preprocessing	10	11	pandas statistics, or data that is cross-validated
Overlap / duplicates	7	2	resampling before the split has no automatic fix
Multi-test	0	5	nested validation is not automated
Group (entity)	7	0	
Temporal	7	18	forward-looking features and whole-series statistics have no automatic fix
Target-derived	3	0	
All types	**34**	**36**	

Accuracy on TEST (files with a single leak, averaged over data seeds 0 to 4)

Leakage type	Metric	Findings	Mean true inflation	Mean estimate	Mean absolute error	Direction right
Preprocessing	test_auc	8	-0.0003	+0.0001	0.0003	60%
Overlap / duplicates	cv_r2	1	+0.0259	+0.0259	0.0000	100%
Overlap / duplicates	test_auc	5	+0.1969	+0.1969	0.0000	100%
Group (entity)	cv_auc	3	+0.0753	+0.0706	0.0047	100%
Group (entity)	test_auc	3	+0.2119	-0.0840	0.2959	0%
Temporal	cv_auc	3	+0.0047	+0.0047	0.0000	100%
Temporal	test_mae	3	+1.3322	+1.3322	0.0000	100%
Target-derived	cv_auc	1	+0.4605	+0.4605	0.0000	100%
Target-derived	cv_r2	1	+0.0475	+0.0475	0.0000	100%
Target-derived	test_auc	1	+0.4433	+0.4433	0.0000	100%

Over the 26 TEST estimates on bounded metrics (AUC, R2) the mean absolute error is 0.035 and the rank correlation with the true inflation is 0.58 (DEV, one seed: 0.027 and 0.96). What these numbers mean:

- **Where the fix is the same operation as the benchmark's own correct version** (overlap, temporal splits, target-derived features), the error is zero. That shows the fixing machinery is right; it is not a hard test.
- **Hold-out group splits fail on TEST** (group, test_auc: estimate -0.08 against a true +0.21, wrong direction). This is the wrong-column problem of section 3.5: the fix grouped by visit number, not by patient. On DEV the same kind of estimate had the right direction but was off by 0.30 for a different reason (few sites, different sites held out). Group impact under cross-validation is accurate on both splits.
- **Preprocessing leaks barely move the score** (true inflation about 0.000 on TEST), which agrees with [5]; the sign of such a small number is noise.
- **Coverage is partial:** an estimate for 34 of 70 located leaks on TEST. The rest are reported as "impact unknown" with the reason, not dropped. LeakHound's proxy estimate, the baseline for this task, has not been run.
- Injected inflation is built into our synthetic data. It is used only to check the estimates; it is not evidence of how severe any leakage type is in practice.

3.8 What went wrong, and what is missing

- **TEST exposed three faults** that DEV did not: a positional slice mistaken for a split, an unresolved column filter, and a weak guess at the entity column that the run confirmed instead of correcting (section 3.5). They are recorded in EVAL_CHANGES.md and are not fixed.
- **CalibraEEG case study: LeakLens found nothing.** We ran the static analyser on the companion project's 78 Python files. Expected: leakage flagged in regime A (random windows) and B (within-patient), nothing in C. Result: 0 findings in all three. That is correct for C and a miss for A and B: this code splits with hand-written NumPy masks over custom recording objects, not with scikit-learn splitters on data frames, which is all the analyser models today. The motivating example of Chapter 1 is not yet detected in its own code base.
- **The name is taken.** The novelty search found an unrelated package called `leaklens` on PyPI (first release June 2026, by another author): data-level checks plus a simple syntax check for preprocessing before a split. It does not detect group leakage from code and does not estimate impact, so the contributions stand, but the project needs a decision on its name before any release or paper.
- **A related idea exists for impact.** Leakly (2026) re-runs a user's pipeline on shuffled labels to test for leakage. It does not locate a leak or apply a fix, but our claim must be worded as per-finding impact, and Leakly must be cited.
- **Still to do:** human labels for real notebooks and their agreement, the other baselines (scicode-lint, the data-level tools, LeakHound, splitguard), the mining study, packaging. One test-split detail was seen by accident while the rules were being written (column names of the TEST pipelines); it is logged.
- **The benchmark and the detector have one author.** TEST uses other pipelines but the same kinds of injected leak, so TEST does not show how the tool copes with leaks written by other people. Only the human-labelled real notebooks can show that.

3.9 Comparison with existing systems

First, capabilities, read from each tool's paper or documentation (Table 2.2 extended with the tools found since Chapter 2, and "proposed" replaced by what is implemented).

System	Reads	Types detected	Group leakage	Temporal leakage	Impact estimate
Yang et al. [3]; LeakageDetector [6][13]	code	preprocessing, overlap, multi-test	no	no	no
LeakCheck, Draoui et al. [7]	code + LLM	preprocessing, overlap	not reported	not reported	no
scicode-lint [15]	code + LLM	66 patterns incl. two temporal	no	shuffled CV, forward shifts	no
splitguard [16]	run	overlap, fit on held-out rows	only if the user declares group ids	no	no
Deepchecks [8], leakcheck-ml, LeakHound, dataleaks, leaklens (PyPI) [17]	data tables the user exports	duplicates, date overlap, some group checks	some, column usually named by user; no code location	date overlap on given splits	LeakHound: its own proxy model
Leakly [18]	re-run on shuffled labels	any (not located, not typed)	no	no	indirect, whole pipeline
LeakLens (this work, implemented)	**code, then a run**	**all six, with line and suggested fix**	**yes: from file paths or names in code; from the data at run time with nothing declared (column choice not yet reliable, 3.5)**	**yes: split and feature level, by data flow; confirmed on the data**	**yes, per finding, by re-running the user's script with the fix (6 kinds of fix)**

"no" and "not reported" mean the source we read does not state support. Tools known only from PyPI were read from package metadata and README (literature/related_work.csv).

Measured: Yang et al.'s analyser and LeakLens on the same TEST files, per file and type

Second, one measured comparison. The reference analyser [3] was built from source on Kaggle and run with its default command on our benchmark files (notebooks converted to scripts, as its README requires). It reports the line of the affected evaluation call, not the leaking line, so both tools are scored here per file: does the tool report this leak type in a file that has it. LeakLens is in static mode, like the baseline.

Leakage type	Files with this leak	Yang et al. [3]: files found	Yang: false alarms	LeakLens (static): files found	LeakLens: false alarms
Preprocessing	22	10	1	21	0
Overlap / duplicates	9	7	0	7	0
Multi-test	6	3	0	5	0
Group (entity)	7	0 (not a type it claims)	0	3	0
Temporal	24	3 (not a type it claims)	0	19	0
Target-derived	4	0 (not a type it claims)	0	3	0

Neither tool was tuned on these files, but they are generated by this project with the kinds of leak LeakLens was built for, and their style may differ from the notebooks [3] was designed on, so this favours LeakLens. Numbers that [3] reports on its own data (precision 97.6%, recall 67.8%) are a different measurement and are not comparable with this table.

3.10 Objectives of Chapter 1

#	Objective	Status on 12 November
O1	Taxonomy of six types with a rule per type	Met (docs/TAXONOMY.md), with runnable examples that are part of the test suite.
O2	Hybrid static + dynamic analyser with line, type and fix	Built; evaluated on held-out injected files. Not yet evaluated on real notebooks.
O3	Impact of each finding by fixing and re-running	Built for six kinds of fix; an estimate for 34 of 70 located TEST leaks; wrong for hold-out group splits (3.7).
O4	Labelled benchmark; precision of at least 0.90 and recall above 67.8% on shared types	Benchmark built. On injected TEST, static mode: precision 1.00; recall preprocessing 0.95, overlap / duplicates 0.78, multi-test 0.83. The point estimates meet the target, but the intervals are wide (for example overlap 0.44 to 1.00), the files are our own, and 67.8% was measured on other data. **Met on our injected benchmark only.**
O5	Empirical study on public notebooks	Not started. 150 notebooks sampled; none labelled.
O6	Usable software	CLI, JSON, SARIF and exit codes done. No PyPI release, pre-commit hook or GitHub Action yet.

3.11 Next steps and decisions needed

- **Decide on a second look at TEST.** The three faults found on TEST have obvious fixes. Fixing them and re-running TEST is allowed only as a logged second look, reported next to the first result, never instead of it.
- **Label real notebooks:** two people, in the prepared random order, without seeing the tool's output. This is the only route to a result on real code.
- **Decide the project name** in the light of the existing PyPI package.
- Run the remaining baselines: data-level tools on the splits the instrumented run materialises, LeakHound for impact, scicode-lint for temporal leakage.
- Extend the analyser to hand-written splits (NumPy masks, custom loaders) so that the CalibraEEG case is found; at run time, check every candidate entity column instead of trusting the static guess; add fixes for the leak kinds that have none.

References

[1] S. Kaufman et al. Leakage in data mining: formulation, detection, and avoidance. ACM TKDD 6(4), 2012. https://doi.org/10.1145/2382577.2382579

[2] S. Kapoor, A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. Patterns 4(9):100804, 2023. https://doi.org/10.1016/j.patter.2023.100804

[3] C. Yang, R. A. Brower-Sinning, G. Lewis, C. Kaestner. Data leakage in notebooks: static detection and better processes. ASE 2022. https://doi.org/10.1145/3551349.3556918

[5] S. Roth. Which leakage types matter? A quantitative landscape across 2,047 benchmark datasets. arXiv:2604.04199, 2026.

[6] E. A. AlOmar et al. LeakageDetector: an open source data leakage analysis tool in machine learning pipelines. 2025. https://arxiv.org/abs/2503.14723

[7] T. Draoui, M. W. Mkaouer, C. Newman. Improving data leakage detection in machine learning notebooks through static slicing and structured LLM prompts. ACM, 2026. https://doi.org/10.1145/3808199

[8] S. Chorev et al. Deepchecks: a library for testing and validating machine learning models and data. JMLR 23(285), 2022.

[13] LeakageDetector 2.0: analyzing data leakage in Jupyter-driven machine learning pipelines. 2025. https://arxiv.org/abs/2509.15971

[15] Samsonau. scicode-lint: detecting methodology bugs in scientific Python code with LLM-generated patterns. arXiv:2603.17893, 2026.

[16] T. Aiello. splitguard 0.1.0. PyPI, 2026. https://github.com/Tommasoaiello13/splitguard

[17] Prajwal. leaklens 1.3.0 (unrelated package of the same name). PyPI, 2026. https://pypi.org/project/leaklens/

[18] DeMONLab-BioFINDER. Leakly 0.1.2. PyPI, 2026. https://github.com/DeMONLab-BioFINDER/Leakly

[1] to [14] as in Chapters 1 and 2. [16] to [18]: package metadata and README only. Result numbers are read from results/2026-11-12_test_injected_leaklens_v1 and results/2026-11-12_dev_injected_leaklens_v2.